

Deployment Portfolio Task 1

Desktop deployment using the Windows Installer XML (WiX) tool

1. Student details:

- **Student ID:** 105559691
- **Full name:** Dang Duc Minh

2. Unit details:

- **Unit code and Name:** SWE40006 Software Deployment and Evolution
- **Semester:** Semester 3, 2026

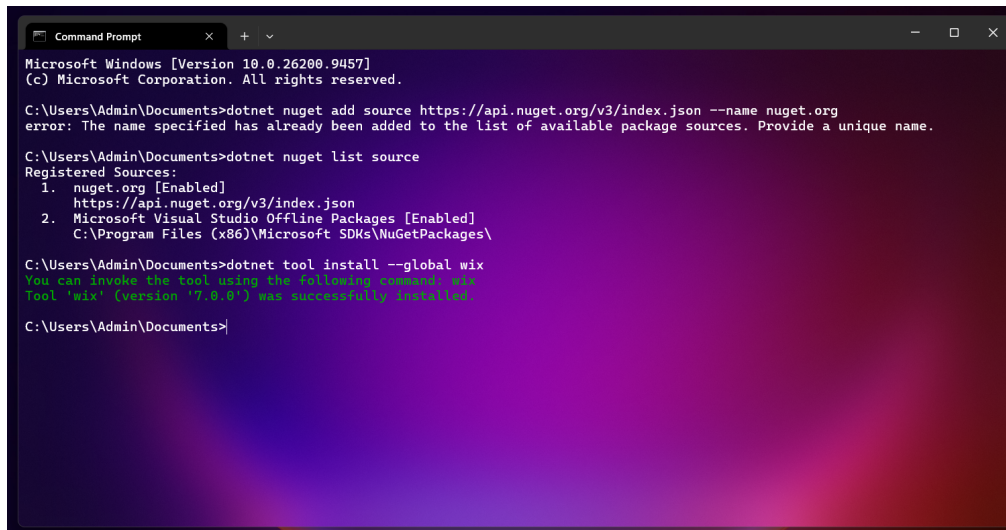
3. Due date and submission date:

- **Due date:** 28 September 2026, 23:59
- **Submission date:** 28 September 2026

4. Declaration of the task level attempted:

- **Level Attempted:** High Distinction (Sub-tasks 1.1 to 1.4)
 - **Source code:**
 - **Task 1.1 (SampleApp):** <https://github.com/mikedeveloper/SWE40006/tree/master/task1/task1.1/SampleApp>
 - **Tasks 1.2 to 1.4 (AlphatechDSP 2.0):** <https://github.com/mikedeveloper/AlphatechDSP-2.0>
-

Task 1.1 (Pass):



```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\Documents>dotnet nuget add source https://api.nuget.org/v3/index.json --name nuget.org
error: The name specified has already been added to the list of available package sources. Provide a unique name.

C:\Users\Admin\Documents>dotnet nuget list source
Registered Sources:
1. nuget.org [Enabled]
   https://api.nuget.org/v3/index.json
2. Microsoft Visual Studio Offline Packages [Enabled]
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

C:\Users\Admin\Documents>dotnet tool install --global wix
You can invoke the tool using the following command: dotnet wix
Tool 'wix' (version '7.0.0') was successfully installed.

C:\Users\Admin\Documents>
```

Figure 1: Installing the WiX toolset globally via the CLI.

To get started, the development environment needed to be set up for WiX. Using the CLI, the NuGet package source was checked and the WiX toolset was installed globally with `dotnet tool install --global wix`, which pulled in version 7.0.0.

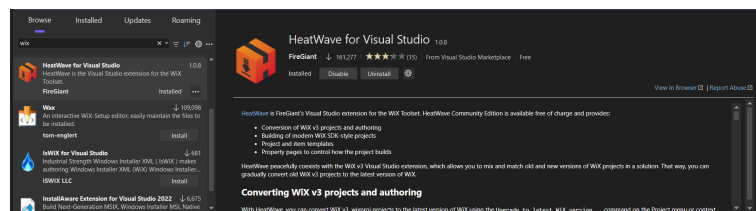


Figure 2: HeatWave for Visual Studio extension installed and enabled.

The next thing was to set up WiX on Visual Studio. The **HeatWave for Visual Studio** extension by FireGiant was installed from the Visual Studio Marketplace. This adds WiX project templates and build support directly to the Visual Studio IDE.

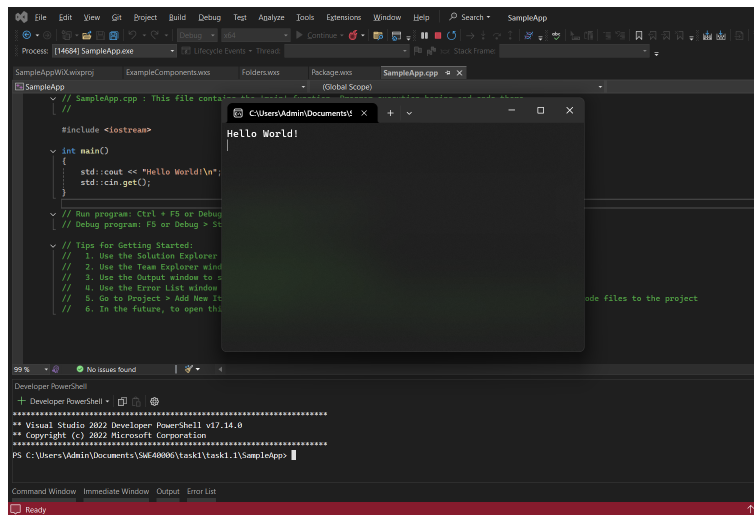


Figure 3: SampleApp compiled and running successfully in Visual Studio 2022.

The sample application, **SampleApp**, is a simple C++ console program that prints “Hello World!” (can be found in this GitHub repository: <https://github.com/mikedeelman/SWE40006/tree/master/task1/task1.1/SampleApp>). The project was built and run to make sure everything works before packaging it.

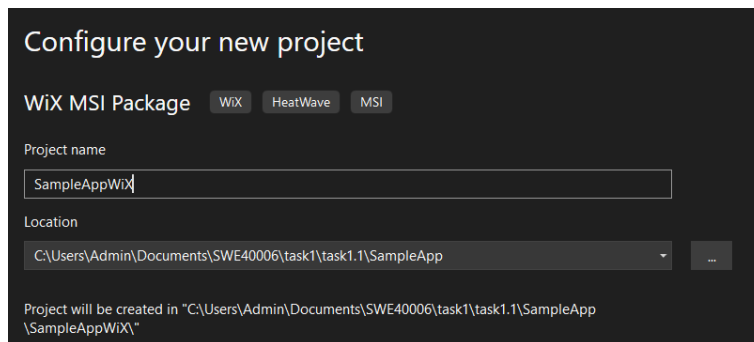


Figure 4: Configuring the new WiX MSI Package project named SampleAppWiX.

To create the installer project, a new **WiX MSI Package** project was added to the solution via HeatWave. The project was named **SampleAppWiX** and placed inside the existing **SampleApp** solution directory so that both projects live under the same solution.

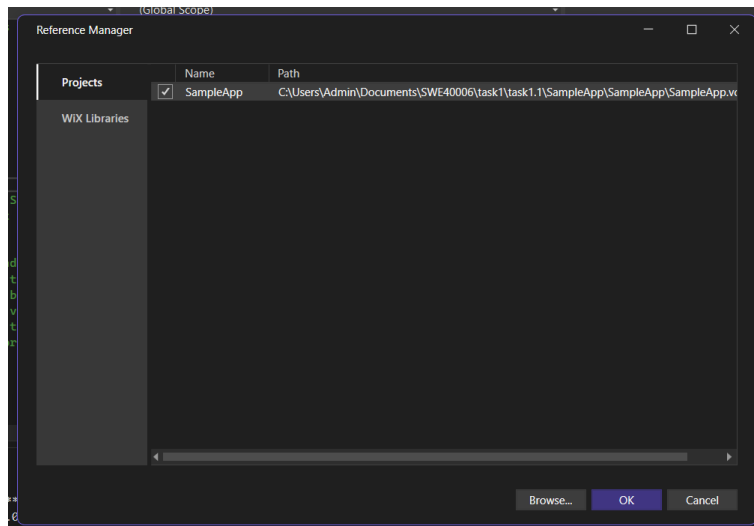


Figure 5: Adding a project reference from SampleAppWiX to SampleApp.

A project reference was then added from `SampleAppWiX` to `SampleApp` through the Reference Manager. This link lets the WiX build process automatically find the compiled `SampleApp.exe`, so the installer always packages the latest build.

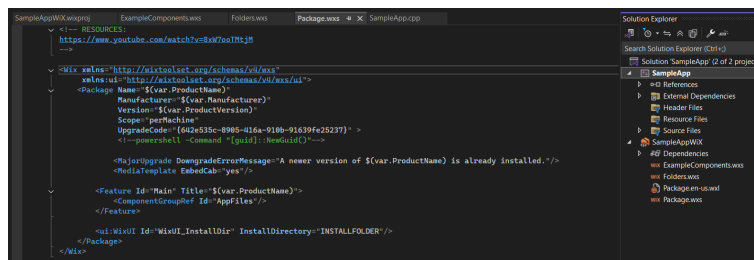


Figure 6: The authored `Package.wxs` manifest.

The main installer configuration lives in `Package.wxs`. This file sets the product name, an upgrade code for version management, per-machine scope attribute, a `MajorUpgrade` element to block downgrades, and a `MediaTemplate` to embed the cabinet files (compressed archives holding the application binaries). A `Feature` element groups the app files, and a `WixUI` reference adds the standard setup wizard dialogs.

`ExampleComponent.wxs` and `Folders.wxs` files were also configured to define the installation folder inside Program Files and specify the executable file to deploy along with its shortcuts.

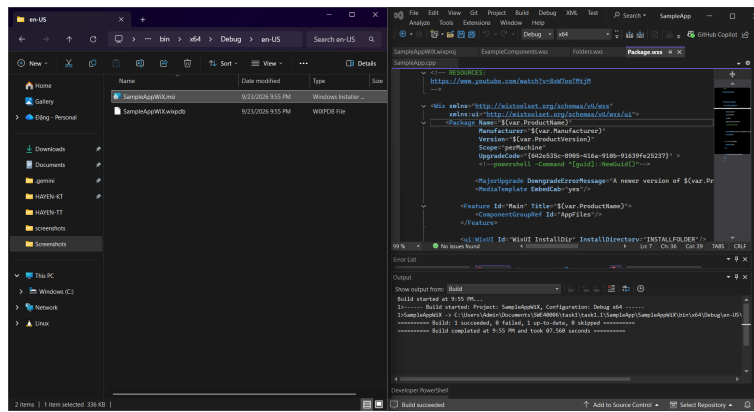


Figure 7: Successful build producing **SampleAppWiX.msi** in the output directory.

Building the **SampleAppWiX** project created the final **SampleAppWiX.msi** installer in the output folder.

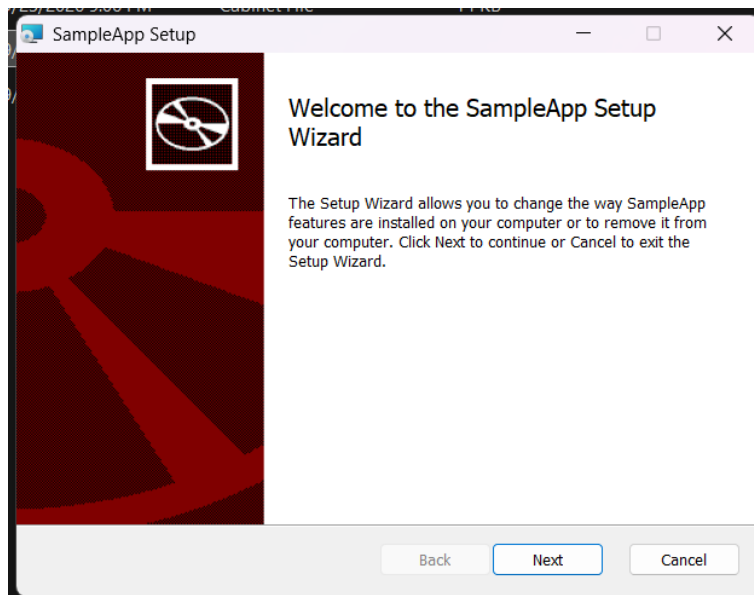


Figure 8: The **SampleApp** Setup Wizard.

Double-clicking the MSI file opened the Windows Installer setup wizard. The dialog shows the product name and gives users options for program installation configuration.

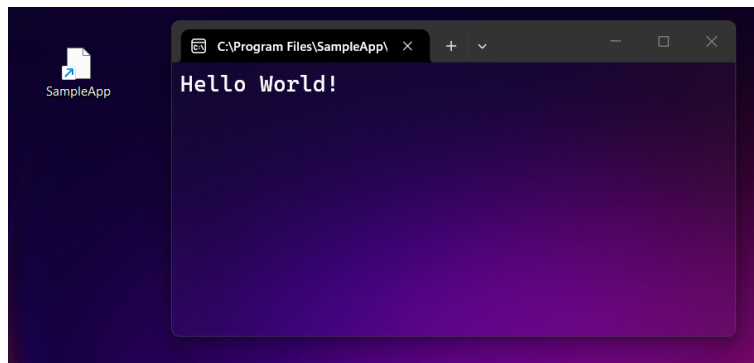


Figure 9: SampleApp running from the installed Program Files directory.

After finishing the installation, the application was placed in `C:\Program Files\SampleApp`. Opening the executable file showed the console prints “Hello World!” and the application shortcut is visible on the desktop.

Task 1.2 (Credit):

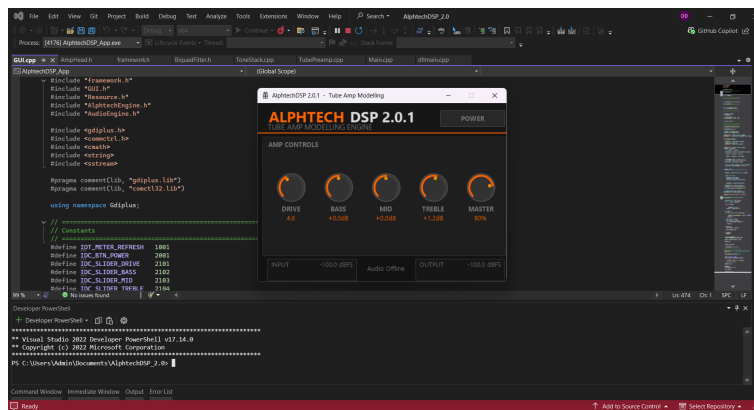


Figure 10: AlphtechDSP 2.0 running in Visual Studio 2022.

With the basic WiX workflow done in Task 1.1, the next step was to apply the same process to a custom application. The target here is **AlphtechDSP 2.0**, my personal C++ project that models a virtual guitar preamp (can be found in this GitHub repository: <https://github.com/mikedevman/AlphtechDSP-2.0>). It includes real-time audio I/O Digital Signal Processing (DSP), with adjustable drive gain and a 3-band EQ. The application was built and run to make sure everything works before packaging.

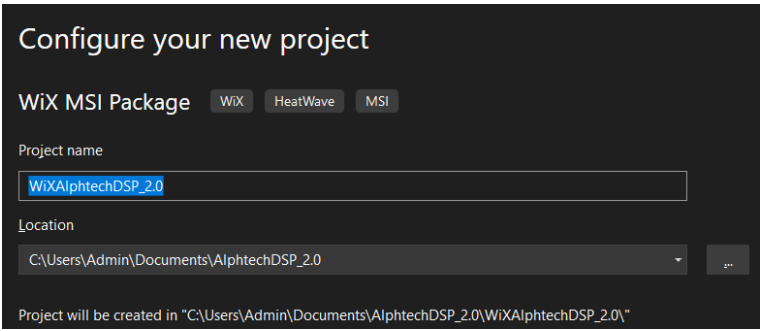


Figure 11: Configuring the new WiX MSI Package project named WiXAlphtechDSP_2.0.

A new **WiX MSI Package** project was added to the AlphtechDSP solution using the same HeatWave template from Task 1.1. The installer project was named **WiXAlphtechDSP_2.0** and placed inside the existing solution directory.

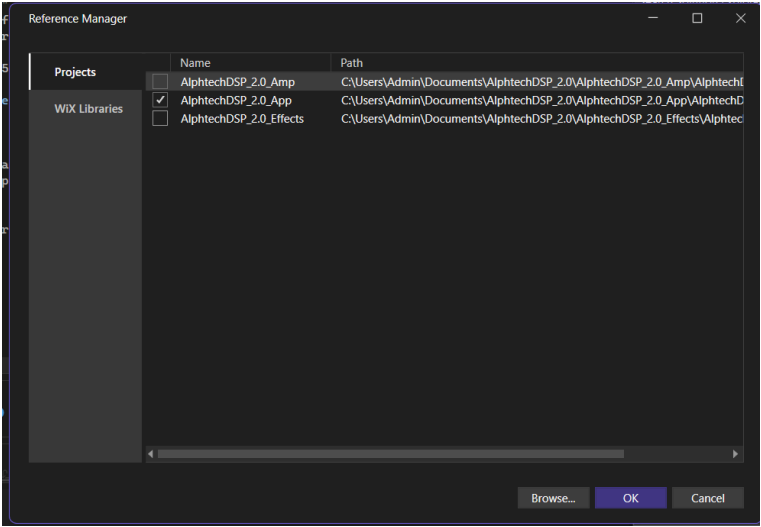


Figure 12: Adding a project reference to AlphtechDSP_2.0_App only.

The Reference Manager was opened to link the installer to the application. The solution has three projects: **AlphtechDSP_2.0_Amp**, **AlphtechDSP_2.0_Effects**, and **AlphtechDSP_2.0_App**. For this initial deployment, only the main application project **AlphtechDSP_2.0_App** was selected. The two DLL projects were left unchecked on purpose to show what happens when dependencies are missing.

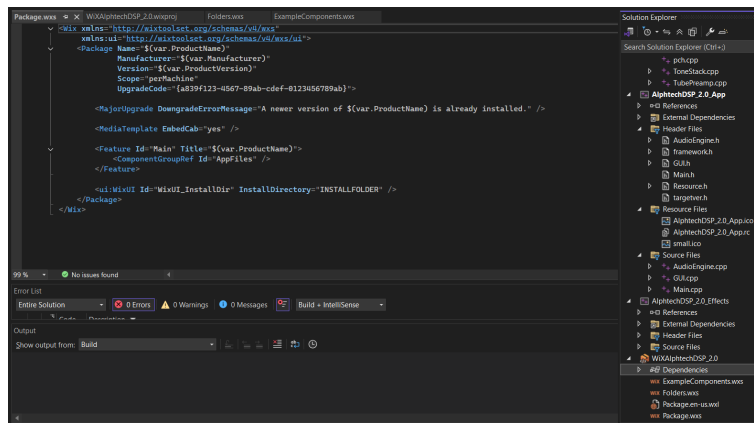


Figure 13: The authored Package.wxs manifest.

The WiX manifest files were then written. Package.wxs was set up with the product name, manufacturer, version, and an upgrade code. The WixUI_InstallDir dialog set was included to give the user a full setup wizard with folder selection. The Folders.wxs and ExampleComponents.wxs files were also configured alongside it.

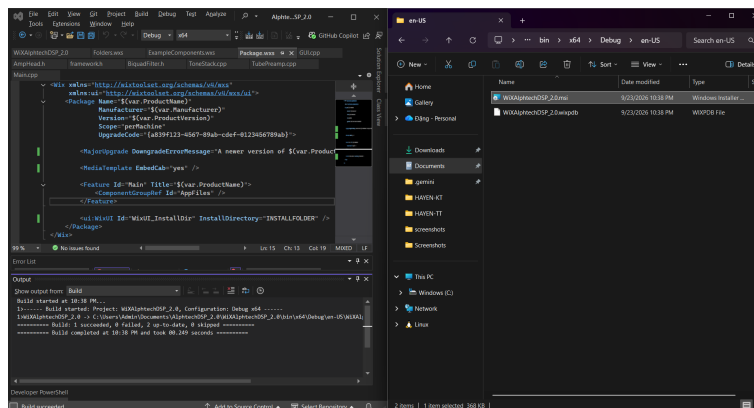


Figure 14: Successful build producing WiXAlphtechDSP_2.0.msi in the output directory.

Building the installer project created WiXAlphtechDSP_2.0.msi in the output folder. The output terminal showed a clean build with 1 succeeded and 0 failed.

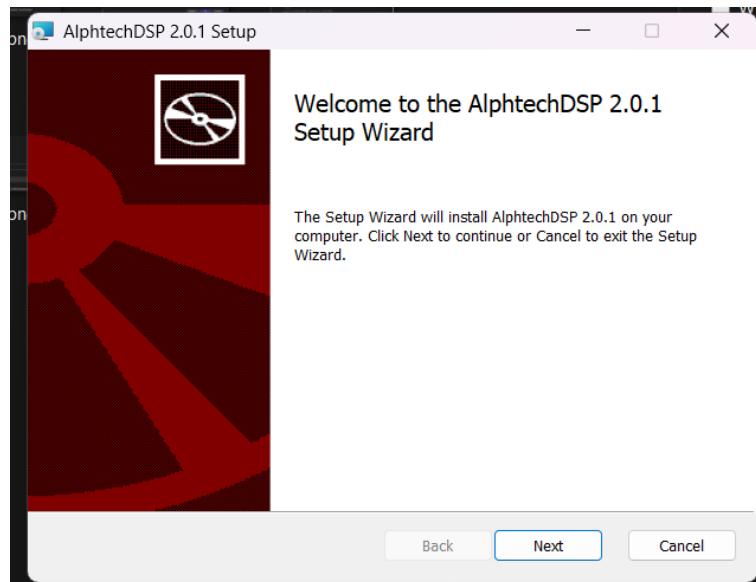


Figure 15: The AlpthechDSP 2.0.1 Setup Wizard launched from the generated MSI.

Running the MSI opened the AlpthechDSP 2.0.1 Setup Wizard, showing that the installer metadata were set up correctly.

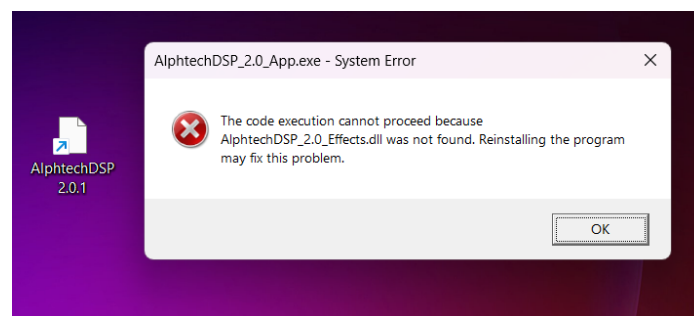


Figure 16: System error: AlpthechDSP_2.0_Effects.dll was not found.

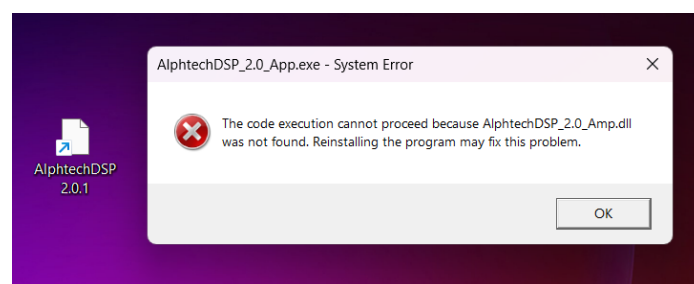


Figure 17: System error: AlpthechDSP_2.0_Amp.dll was not found.

After the installation finished and the application was launched from the desktop shortcut, a system error popped up right away. Windows said that `AlpthechDSP_2.0_Effects.dll` and `AlpthechDSP_2.0_Amp.dll` were not found, which stopped the application from starting. This happened because only the executable was packaged and the shared DLL files were not included in the installer. This confirmed that the application needs both DLL files to run and this failure leads to Task 1.3, where both DLLs will be added to the installer.

Task 1.3 (Distinction):

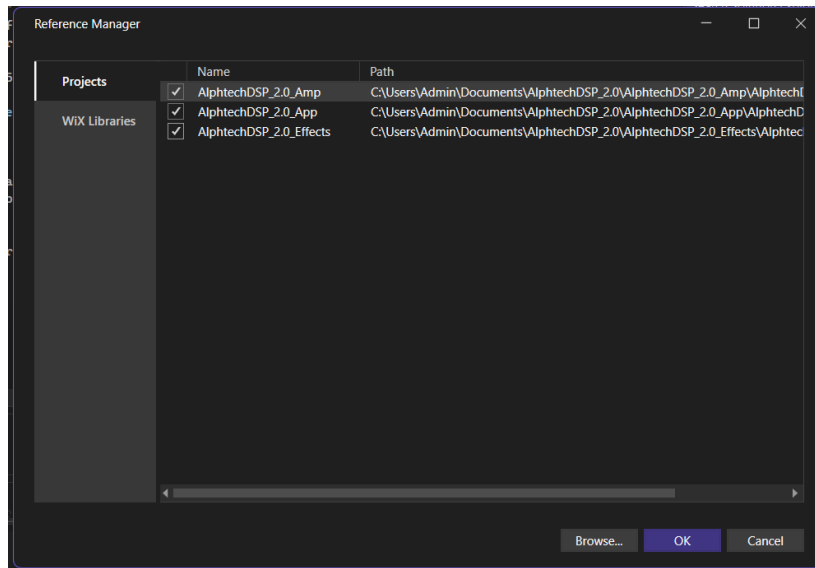


Figure 18: All three project references now checked in the Reference Manager.

In order to fix the missing DLL files errors in Task 1.2, the installer needed to include all the runtime files. The first step was to go back to the Reference Manager and check all three projects: AlphtechDSP_2.0_App, AlphtechDSP_2.0_Amp, and AlphtechDSP_2.0_Effects, so the WiX installer can find every output binary.

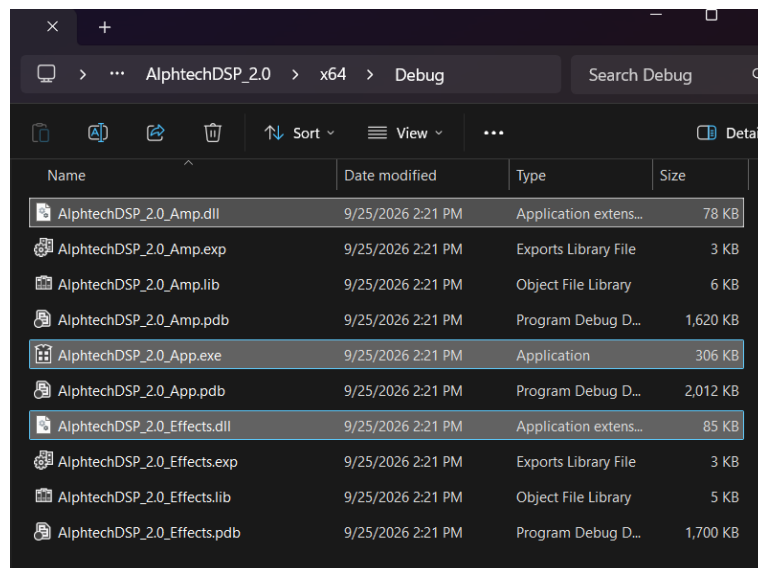


Figure 19: Build output directory showing the executable and both DLLs.

The solution was rebuilt to generate the missing libraries. Both AlphtechDSP_2.0_Amp.dll and AlphtechDSP_2.0_Effects.dll had been produced next to AlphtechDSP_2.0_App.exe in the x64\Debug output folder.

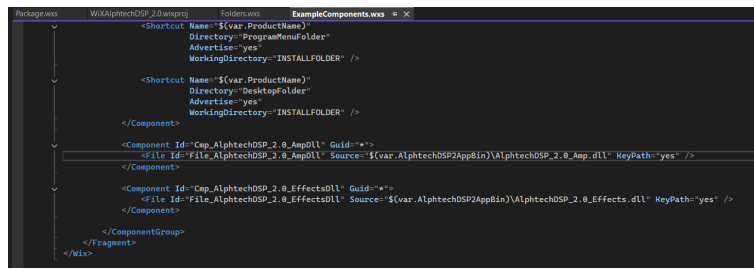


Figure 20: Updated ExampleComponents.wxs with both DLL components declared.

The ExampleComponents.wxs manifest was then updated to add a separate Component for each DLL. Below the main executable component, two new components (AlphtechDSP_2.0_Amp.dll and AlphtechDSP_2.0_Effects.dll) were added. Both go into the same INSTALLFOLDER as the executable so Windows can find them when the app starts.

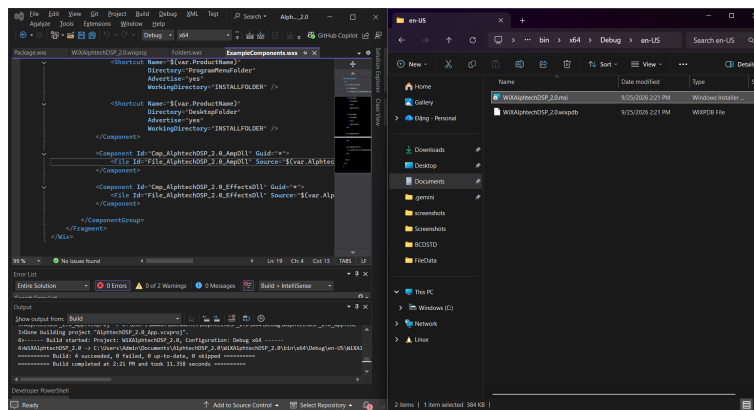


Figure 21: Successful build of all four projects and the updated MSI in File Explorer.

Rebuilding the solution now compiled all four projects: the three application projects and the WiX installer. The output terminal showed 4 succeeded and 0 failed, and the updated WixAlphtechDSP_2.0.msi appeared in the output folder.

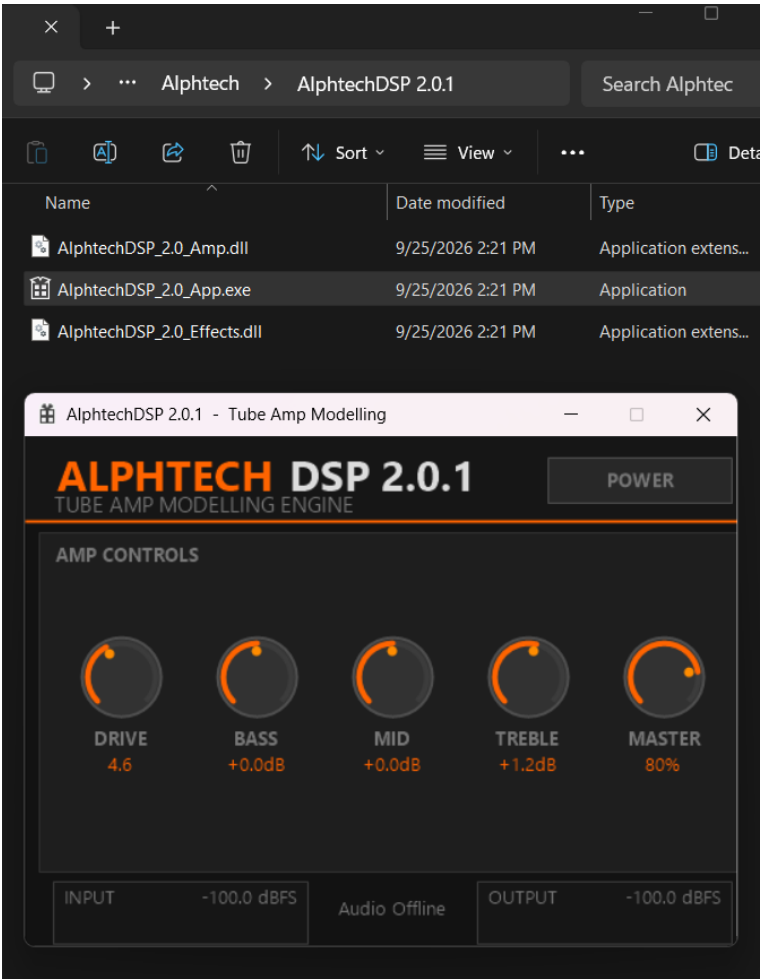


Figure 22: AlphtechDSP 2.0 ran successfully with all dependencies resolved.

After running the updated installer, the folder at `Program Files\Alphtech\AlphtechDSP 2.0.1` now has all three files: the executable and both DLLs sitting together. Launching the application opened the AlphtechDSP 2.0 interface immediately without any missing DLL errors, which confirms that including the DLLs in the installer fixed the issues from Task 1.2.

Task 1.4 (High Distinction):

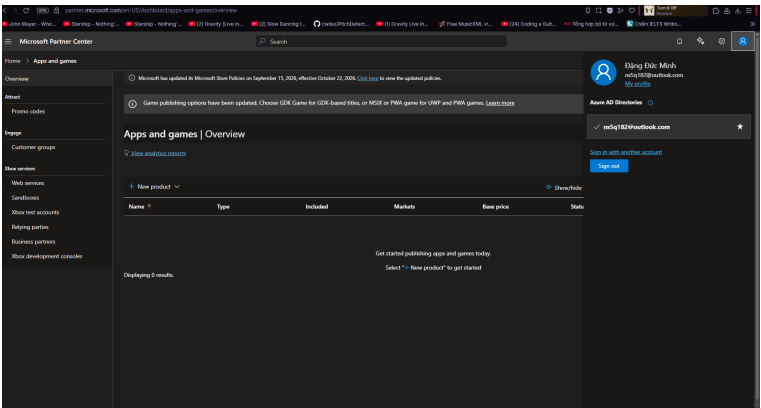


Figure 23: Logging into Microsoft Partner Center and setting up a developer profile.

I started this task by logging into the Microsoft Partner Center and setting up my developer profile.

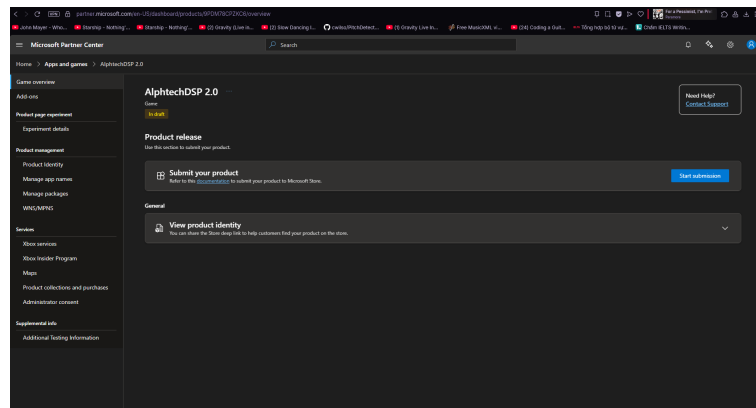


Figure 24: Reserving the app name AlphtechDSP 2.0.

Once my profile was ready, I created a new app submission and reserved the name **AlphtechDSP 2.0** for the project.

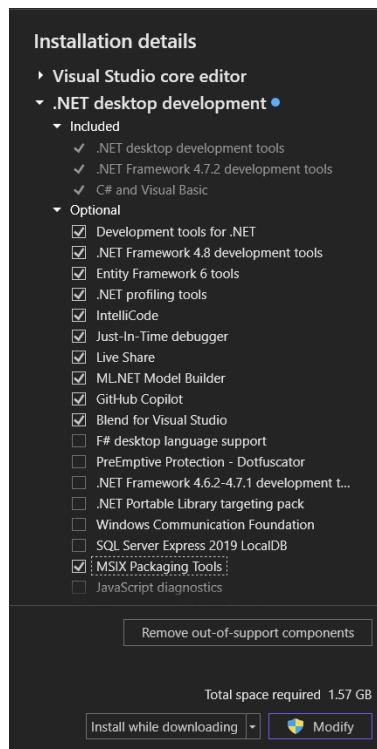


Figure 25: Installing the MSIX Packaging Tools in the Visual Studio Installer.

To get Visual Studio ready for packaging, I opened the Visual Studio Installer and made sure the MSIX Packaging Tools component was checked and installed.

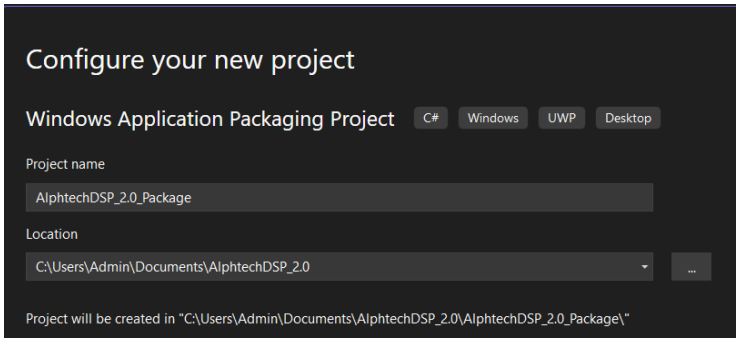


Figure 26: Adding a new Windows Application Packaging Project to the solution.

Back in my solution, I added a new Windows Application Packaging Project and named it AlphtechDSP_2.0_Package.

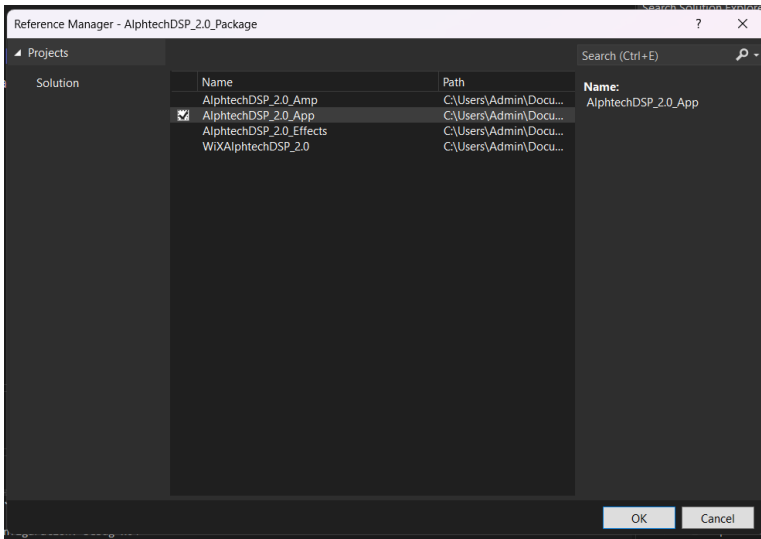


Figure 27: Adding a project reference to AlphtechDSP_2.0_App.

I then opened the Reference Manager for the new packaging project and ticked the box for AlphtechDSP_2.0_App. Referencing the main application ensures that all of its project dependencies and DLL files are automatically harvested and included in the package too.

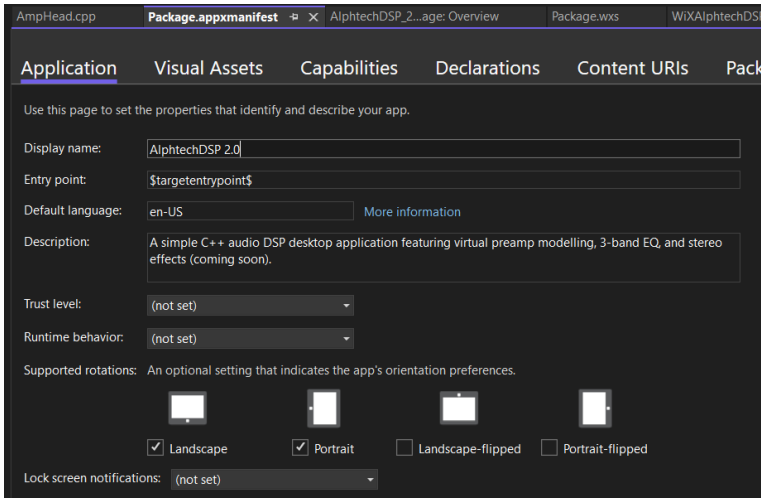


Figure 28: Opening the package manifest to configure app details.

Next I opened the package manifest to set up the app details such as display name and description.

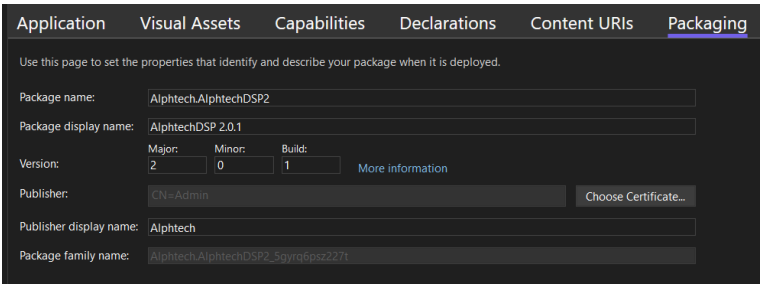


Figure 29: Updating the package display name and version number in the Packaging tab.

In the Packaging tab of the manifest, I updated the package display name and set the version number to 2.0.1.0 to match the current build.

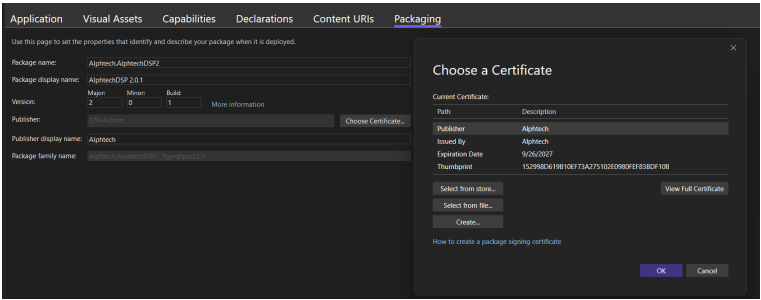


Figure 30: Generating a local test certificate for package signing.

I also generated a local test certificate in the same tab so the package could be signed during the build process.

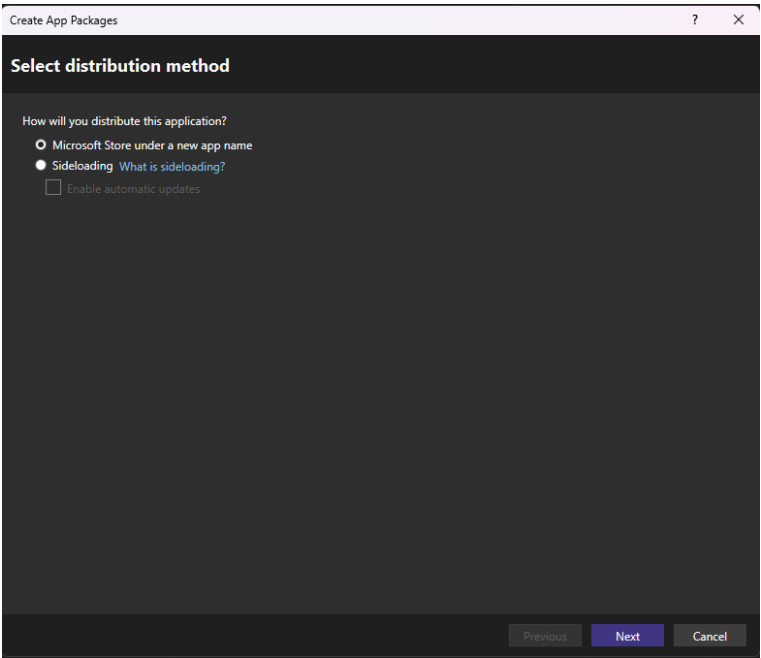


Figure 31: Starting the Create App Packages wizard to distribute to the Microsoft Store.

With the configuration done, I right clicked the packaging project and started the Create App Packages wizard. I chose the option to distribute to the Microsoft Store under a new app name.

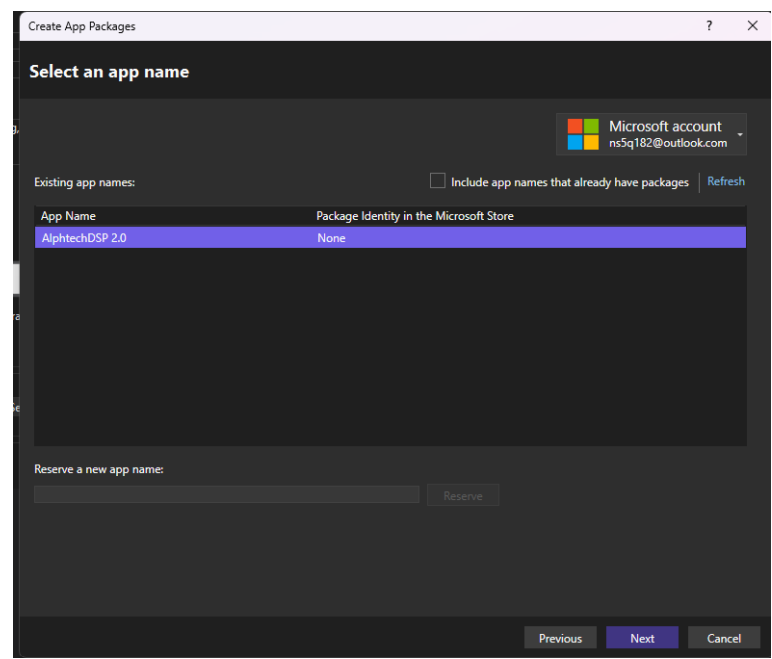


Figure 32: Selecting the reserved app name after signing in with the Microsoft account.

The wizard prompted me to sign in with my Microsoft account. After logging in, it found the AlphtechDSP 2.0 name I reserved earlier and I selected it.

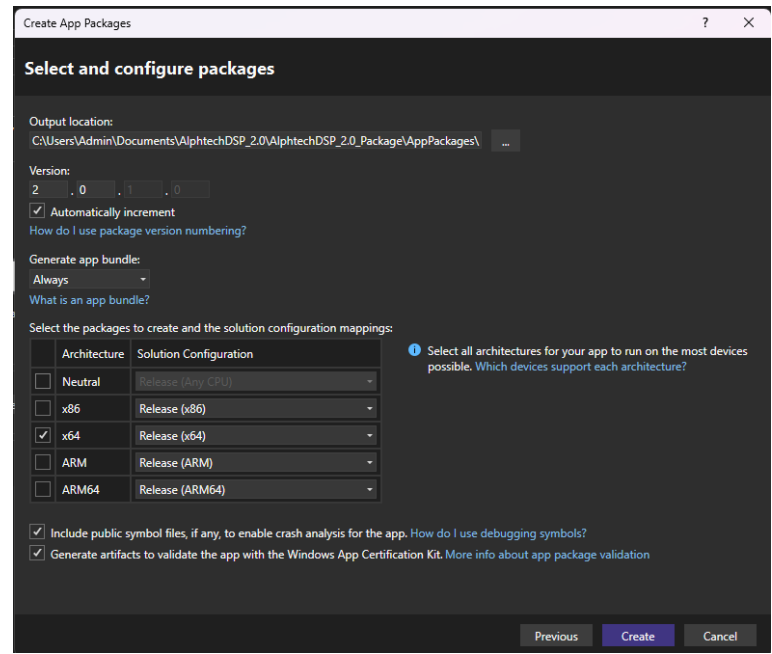


Figure 33: Configuring the package architecture for x64 only.

I then configured the package architecture. I unchecked the neutral option and specifically checked x64 to ensure it builds correctly for modern processors.

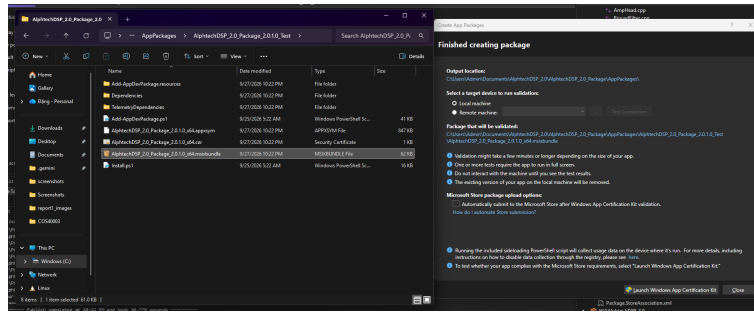


Figure 34: Visual Studio completing the build of the MSIX bundle.

Visual Studio then compiled the project and built the final MSIX bundle. A confirmation window popped up and I opened the output folder to verify the generated package was there.

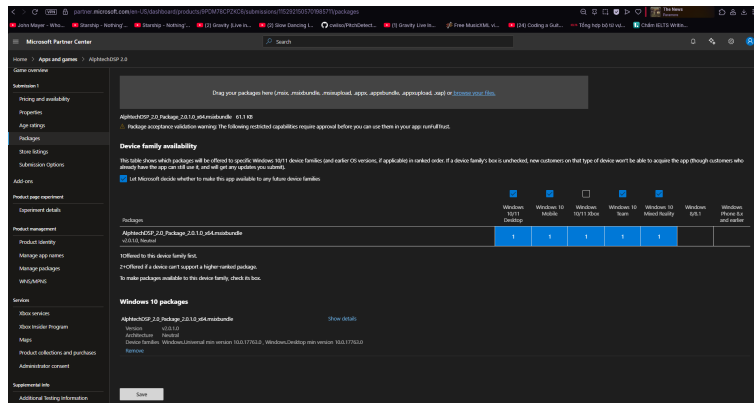


Figure 35: Uploading the MSIX bundle to the for validation.

I returned to Microsoft Partner Center and uploaded the newly created MSIX bundle into the Packages section of the draft submission. After some short processing time, the package was approved.

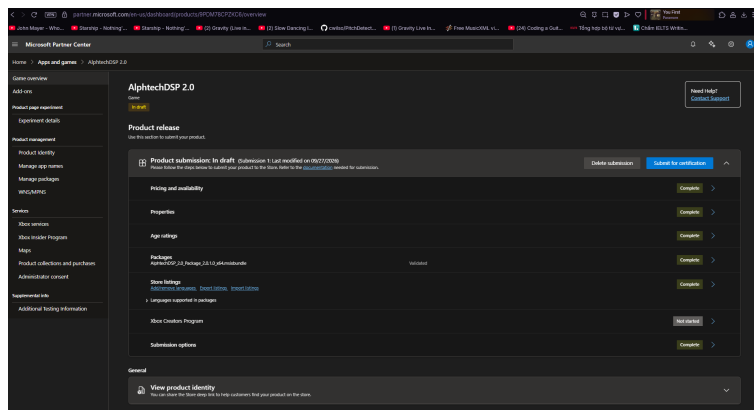


Figure 36: Filling out the store listing details including pricing, properties, and description.

I then filled out the rest of the store listing detail sections. The properties were configured as follows:

- **Pricing and availability:** Set to Free (0 USD).
- **Properties:** Categorized under the Music genre and linked the GitHub privacy policy (<https://github.com/mikedevean/AlphtechDSP-2.0/blob/master/PRIVACY.md>).
- **Store listings:** Added a description and a screenshot of the application.

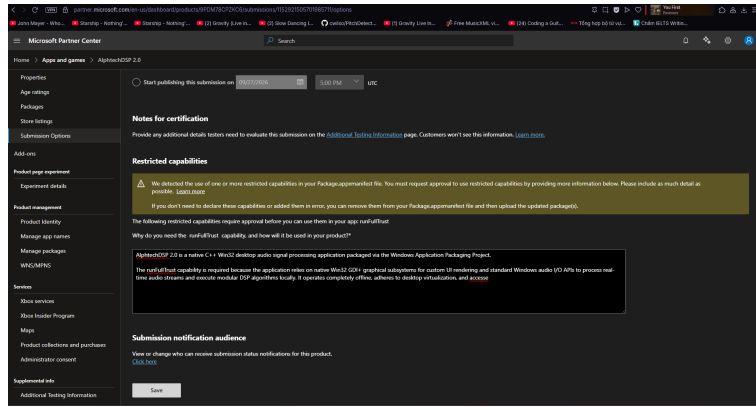


Figure 37: Providing justification for the `runFullTrust` capability.

Because this is a native C++ Win32 application, it requires the `runFullTrust` capability. In the **Submission options** section, I had to provide a justification for this, explaining that it needs full trust for real-time audio processing and custom UI rendering using standard Windows APIs. Once all sections were marked complete (except for **Xbox Creators Program**, which is not needed for this application), I was ready to submit.

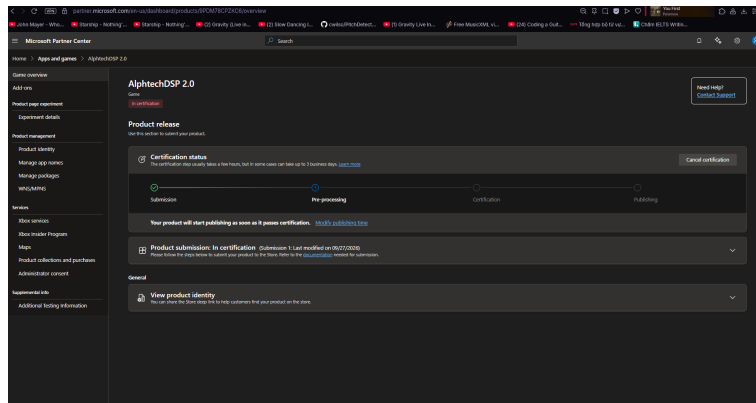


Figure 38: The certification status tracker showing the submission is under processing.

Finally, I clicked the Submit for certification button. The dashboard updated to show the certification status tracker, which marked the submission step as completed, and we will need to wait for the final processing and approval.

At the time of submitting this report, the application is still under processing. If you want to try it out once it is published, you can search for **AlphtechDSP 2.0** directly in the Microsoft Store.

— THE END —